filename. We'll then provide an additional factory method, from_file, that takes a filename and constructs a new Finder from that file's contents:

gems/aaagmnr/lib/aaagmnr/finder.rb

```
module Aaagmnr
  class Finder
    def self.from_file(file_name)
      new(File.readlines(file_name))
    end

    def initialize(dictionary_words)
      @signatures = {}
      dictionary_words.each do |line|
        word = line.chomp
        signature = signature_of(word)
        (@signatures[signature] ||= []) << word
      end
    end

    def lookup(word)
      signature = signature_of(word)
      @signatures[signature]
    end

    def signature_of(word)
      word.unpack("c*").sort.pack("c*")
    end
  end
end
```

Again, we embed the Finder class inside the top-level Aaagmnr module. And, again, this code is self-contained, allowing us to write some simple unit tests:

gems/aaagmnr/spec/aaagmnr/finder_spec.rb

```
module Aaagmnr
  RSpec.describe Finder do
    describe "signature" do
      subject(:finder) { Finder.new([]) }

      specify { expect(finder.signature_of("cat")).to eq("act") }
      specify { expect(finder.signature_of("act")).to eq("act") }
      specify { expect(finder.signature_of("wombat")).to eq("abmotw") }
    end

    describe "lookup" do
      subject(:finder) { Finder.new(["cat", "wombat"]) }

      it "returns the word if the word is given" do
        expect(finder.lookup("cat")).to eq(["cat"])
      end

      it "returns the word if an anagram is given" do
        expect(finder.lookup("act")).to eq(["cat"])
        expect(finder.lookup("tca")).to eq(["cat"])
      end

      it "returns nil if no word matches the anagram" do
        expect(finder.lookup("wibble")).to be_nil
      end
```